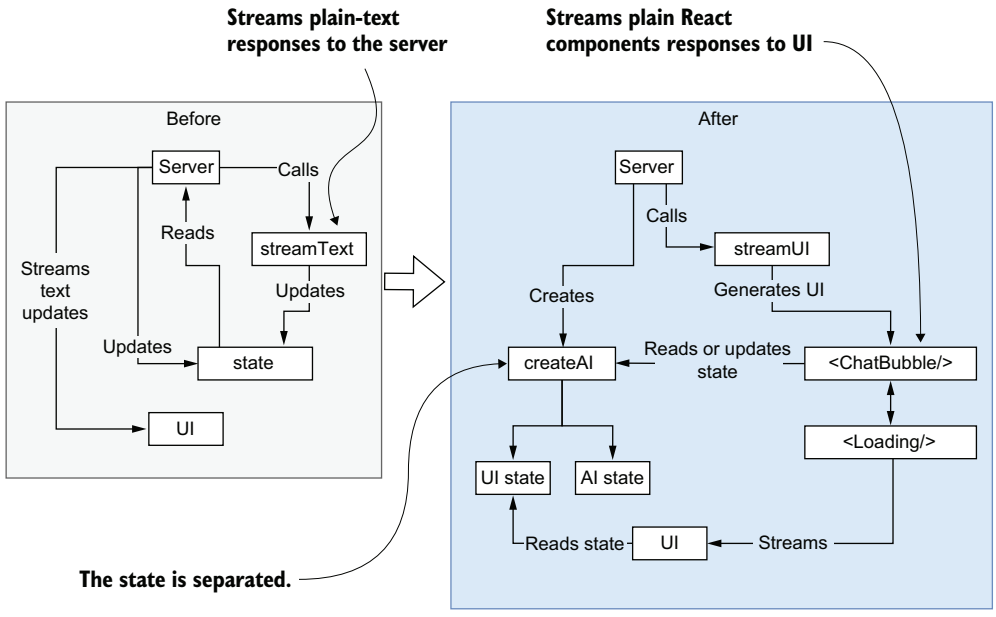


Figure 4.9 The transition from a system with separate management of UI and AI states using `streamText` for plain-text responses to a unified state management approach using `createAI`. In the new architecture, `streamUI` replaces `streamText`, enabling the streaming of React components and providing a consistent context for both AI and UI states through `createAI`. This shift enhances the application's ability to manage content and integrates state management more effectively.

After the upgrade, the `streamUI` function now allows for streaming responses as fully fledged React components instead of plain text. Additionally, the `createAI` function was introduced to unify AI and UI state management. This function provides a single source of truth for the entire application.

Let's take a close look at the implementation, focusing on the server actions where these changes will have the most effect. We'll walk through the transformation step by step, highlighting the key modifications and their implications for our application's architecture. As we proceed, keep in mind that this new approach will not only solve our current challenges but also open doors to more advanced features.

Changes to server actions

To refactor our existing server actions to use these tools, we will change the `continueConversation` action to manage user inputs and conversation history while integrating with `createAI` for consistent state management.

This action now uses `getMutableAIState` to manage the conversation history and `streamUI` to generate a streaming response and updates the AI state accordingly. The `createAI` function sets up the AI context for the entire application, defining the available actions and initial states.